PML Capstone Project Guide — Five Domain Projects

Nov 3, 2026 ·

How to use this guide

Five scikit-learn capstones, one per domain, each built on a free primary-source dataset that a team of 3–4 can finish in six weeks. Every project is a real decision workflow, not a tutorial dataset: the target is something an organisation actually needs predicted, and every feature must exist at the moment the prediction is made.

#	Domain	Project	Data owner	ML task
1	Cybersecurity	Predicting which published CVEs will be exploited in the wild	NIST (NVD) + CISA (KEV)	Binary classification, extreme imbalance
2	Insurance	Flood-claim severity triage	FEMA (OpenFEMA NFIP)	Regression (log payout)
3	Fintech / Banking	Mortgage denial-risk model with a fair-lending audit	CFPB / FFIEC (HMDA)	Binary classification + fairness analysis
4	Healthcare	Screening for undiagnosed prediabetes and diabetes	CDC NCHS (NHANES)	Binary classification, multi-file join
5	Agriculture	Pre-harvest county corn-yield forecasting	USDA NASS + NASA POWER	Regression, time-aware validation

Student profile assumed. Python, pandas, NumPy, EDA, scaling, encoding, imputation, regression and classification with KNN, Decision Trees, Random Forest, SVM, Naive Bayes, ensembles, Gradient Boosting, PCA, cross-validation, GridSearchCV / RandomizedSearchCV, and a basic Streamlit or Flask app. No deep learning, LLMs, paid cloud or proprietary data is needed.

Rules that apply to all five projects.

- **One `Pipeline` object.** Imputation, encoding, scaling and the model live in a single scikit-learn `Pipeline` / `ColumnTransformer`, fitted only on training folds. The same saved object is loaded by the app.
- **Split first, explore second.** Hold out the test set before any statistic is computed that could shape a modelling choice. Look at the test set once, at the end.
- **Time matters.** Where records have dates (projects 1, 2, 3, 5), the final test set is a later time period than the training data. Random splits are allowed only for the cross-sectional survey in project 4.

- **The inference-time test.** For every column, students write one sentence answering: *would this value be known at the moment we make the prediction?* If not, it is dropped or moved to the evaluation-only list.
- **Raw is read-only.** `data/raw/` is never edited by hand. Every processed file is regenerated by a script.
- **Research demo, not a product.** Each app carries a visible banner saying it is a student research prototype and names the human who must make the real decision.

On verification. Dataset pages, field names and access rules below were checked against the official sources on 3 November 2026. Items that could not be confirmed are marked **[verify]**, and students must confirm them in Week 1. No accuracy, F1 or R^2 figures are given anywhere in this guide on purpose: teams must report what their own models achieve.

Project 1 — Cybersecurity: Which new vulnerabilities will attackers actually exploit?

Teams build a model that ranks newly published CVEs by the probability that the U.S. government will confirm them as exploited in the wild within a year, so patching effort goes to the few hundred that matter instead of the tens of thousands published each year.

A. Project definition

Item	Definition
Title	Exploitation-Risk Ranking of Newly Published Vulnerabilities using NVD and the CISA KEV Catalog
Business problem	Security teams cannot patch every CVE. CVSS severity measures impact if exploited, not whether anyone will exploit it, so "patch every Critical" wastes effort and still misses exploited Medium-severity bugs.
Users and stakeholders	Vulnerability-management analysts, SOC leads, IT patch teams, CISOs, managed security service providers
Why it matters	Only a small fraction of published CVEs ever appear in CISA's Known Exploited Vulnerabilities (KEV) catalog, yet these drive real incidents, including ransomware campaigns.
Prediction objective	At the time a CVE is published in NVD, estimate the probability it is added to CISA KEV within 365 days
Target variable	`exploited_365d` = 1 if KEV `dateAdded` − NVD `published` ≤ 365 days, else 0
Inputs at prediction time	CVE description text, CWE weakness ID, CVSS v3.x base-metric vector, affected vendor / product / platform type (CPE), vendor's prior KEV history computed strictly before publication

Item	Definition
Expected business outcome	A ranked daily watch-list: catching more of the eventually-exploited CVEs in the top 5% of the queue than a "CVSS ≥ 9.0" rule does
ML problem type	Binary classification with extreme class imbalance (well under 1% positives); ranking quality matters more than accuracy
Difficulty	Advanced (imbalance, text features, time-aware validation, label caveats)
Six-week feasibility	Feasible. Data pull takes one day with an API key; the hard part is careful labelling and leakage control, which is the learning goal.

B. Data acquisition

Sources.

Dataset	Owner	Access	API key	Size (verified)
Known Exploited Vulnerabilities catalog	CISA (U.S. Cybersecurity and Infrastructure Security Agency)	JSON feed `https://www.cisa.gov/sites/default/files/feeds/known_exploited_vulnerabilities.json`, CSV feed at the same path with `.csv`, and an official GitHub mirror `cisagov/kev-data`	No	1,766 entries in catalog version 2026.10.30
NVD CVE API 2.0	NIST (National Vulnerability Database)	REST endpoint `https://services.nvd.nist.gov/rest/json/cves/2.0`	Optional, free, recommended	NVD's documentation page reports roughly 339,000 CVEs in total

Steps.

1. **KEV.** Download the JSON feed and save it unchanged to `data/raw/kev_<catalogVersion>.json`. Its verified fields are `cveID`, `vendorProject`, `product`, `vulnerabilityName`, `dateAdded`, `shortDescription`, `requiredAction`, `dueDate`, `knownRansomwareCampaignUse`, `forensicTriage`, `notes`, `cwes`. Only `cveID` and `dateAdded` are used for the label.
2. **NVD.** Request a free API key (one per email). The documented limit is 5 requests per rolling 30 seconds without a key and 50 with one. Query by `pubStartDate` / `pubEndDate` in windows of at

most 120 days, page with `startIndex`, and keep `resultsPerPage` at or below the 2,000 maximum. Sleep between calls as NIST recommends. Save every page as raw JSON.

3. **Scope.** Pull CVEs published 1 Jan 2021 to 30 Jun 2025. The cut-off leaves every CVE at least 365 days to be labelled before the KEV snapshot date.
4. **Understand the schema.** Each NVD item holds `cve.id`, `published`, `lastModified`, `vulnStatus`, `descriptions`, `metrics` (`cvssMetricV31`, `cvssMetricV30`, `cvssMetricV40`, each with a `type` of Primary or Secondary), `weaknesses`, `configurations` (CPE match strings) and `references` (URLs with tags).
5. **Authenticity.** Use only `cisa.gov`, `nist.gov` or the `cisagov` GitHub organisation. Record KEV `catalogVersion` and `dateReleased` from the file header.
6. **Join.** Left-join NVD (one row per CVE) to KEV on `cve.id` = `cveID`. Unmatched rows get label 0. Drop CVEs with `vulnStatus` = Rejected.
7. **Storage and provenance.** `data/raw/` (untouched JSON), `data/interim/` (flattened parquet), `data/processed/` (model table). A `DATA_SOURCES.md` records URL, catalog version, NVD extraction timestamps, query windows and the NVD notice: "This product uses the NVD API but is not endorsed or certified by the NVD." KEV and NVD are U.S. government public data; neither contains personal data.

C. Cleaning and data preparation

1. **Missing values.** Many recent CVEs have no NVD-analysed CVSS score or CPE configuration because of NVD's analysis backlog. Fall back to the CNA's Secondary CVSS score where present, then add a `cvss_missing` flag rather than imputing a severity.
2. **Encoded missing.** `NVD-CWE-noinfo` and `NVD-CWE-Other` are placeholders, not real weaknesses: map them to `CWE_unknown`.
3. **Imputation.** Categorical CVSS components: constant `"missing"` category. Numeric base score: median plus indicator, inside the pipeline.
4. **Duplicates.** One row per CVE ID; check that paging did not repeat records at window boundaries.
5. **Inconsistent categories.** Vendor names differ between KEV (`Microsoft`, `Apache`) and CPE strings (`microsoft`, `apache`). Lower-case, strip punctuation and keep a small manual mapping file for the 50 most frequent KEV vendors.
6. **Outliers.** Some CVEs list thousands of CPE matches (e.g. Linux kernel). Cap `n_cpe` at the 99th percentile or log-transform it.
7. **Encoding.** One-hot for CVSS components and platform type; top-N grouping for CWE and vendor; TF-IDF for description text.
8. **Scaling.** Needed only for Logistic Regression and SVM; trees do not need it.
9. **Leakage — the core lesson.**
    - The NVD record carries `cisaExploitAdd`, `cisaActionDue`, `cisaRequiredAction` and `cisaVulnerabilityName` for KEV CVEs. These are the label in disguise: drop them at load time.
    - Reference tags such as `Exploit` are often added after exploitation. Keep them out of the main model and test them only in a labelled "leaky ablation".

- An NVD record downloaded today is its current version, not its version on publication day. Prefer fields that rarely change (description, CWE, CNA vector) and disclose this as a limitation.
- Vendor KEV history must count only entries with `dateAdded` earlier than the CVE's `published` date.

Order of operations. Fixed rules (dropping CISA fields, vendor normalisation, the time-based split) run before the split. Anything learned from data (imputation values, TF-IDF vocabulary, top-N category lists, scaling) is fitted inside the `Pipeline` so it is re-learned on each training fold.

D. Exploratory data analysis

Question	Chart	How it guides feature engineering
How rare are positives, and how fast do they appear after publication?	Histogram of days from `published` to `dateAdded` for positives	Justifies the 365-day window and shows label delay
Does CVSS severity separate exploited from not-exploited?	Box plot of base score by label; stacked bar of KEV rate by severity band	Shows why a severity-only rule is a weak baseline
Which weaknesses are over-represented in KEV?	Bar chart of KEV rate by top-20 CWE	Supports CWE grouping
Are some vendors repeatedly exploited?	Bar chart of KEV rate by top-20 vendors	Motivates prior-KEV-count features
Which words in descriptions differ between classes?	Top TF-IDF terms by class (horizontal bars)	Motivates text features and the "actively exploited" phrase check
Does the positive rate drift over time?	Line chart of monthly KEV rate by publication month	Supports a time-based split and drift monitoring

All EDA is run on the training period only.

E. Feature engineering

Feature	How to calculate	Why it may help	Available at inference?	Leakage risk
CVSS v3.x components	Parse `vectorString` into AV, AC, PR, UI, S, C, I, A	Network-reachable, no-auth bugs are easier to weaponise	Yes, when scored	Low (scores occasionally revised)
`vendor_prior_kev`	Count KEV entries for the normalised vendor with `dateAdded` < CVE	Attackers revisit familiar targets	Yes	High if the date condition is

Feature	How to calculate	Why it may help	Available at inference?	Leakage risk
	`published`			skipped
`vendor_cve_volume_12m`	CVEs for that vendor in the 12 months before publication	Popular, widely deployed software is a bigger target	Yes	Low
`platform_type`	CPE part `a` / `o` / `h` (application, OS, hardware)	Edge devices and OSs are frequent KEV entries	Only after NVD analysis; add a missing flag	Low
Description TF-IDF	`TfidfVectorizer` (1–2-grams, `min_df=5`, ≤5,000 terms) inside the pipeline	Phrases such as "remote code execution" or "authentication bypass" carry signal	Yes	Medium: some vendors state "may have been actively exploited" at publication — legitimate, but students must report how many CVEs this affects
`cwe_group`	Top-25 CWEs kept, rest `other`	Weakness class relates to exploitability	Yes	Low
`n_cpe_log`	log(1 + number of CPE matches)	Breadth of affected products	After analysis	Low

F. Model development

Algorithm	Why appropriate	Scaling	Key hyperparameters	Strengths / limitations
Baselines: `DummyClassifier` and a "CVSS ≥ 9.0" rule	Shows the value added over doing nothing and over current practice	No	—	Mandatory comparison points
Logistic Regression	Interpretable coefficients; handles sparse TF-IDF well	Yes	`C`, `penalty` (l1/l2), `class_weight`	Linear; needs good features

Algorithm	Why appropriate	Scaling	Key hyperparameters	Strengths / limitations
Complement Naive Bayes on TF-IDF	Designed for imbalanced text classification	No (non-negative counts)	`alpha`	Text-only; poorly calibrated probabilities
Random Forest	Non-linear interactions between vendor, CWE and vector	No	`n_estimators`, `max_depth`, `min_samples_leaf`, `class_weight`	Slow on wide sparse text; use structured features plus a few text flags
`HistGradientBoostingClassifier`	Strong on tabular data, native missing-value handling	No	`learning_rate`, `max_leaf_nodes`, `l2_regularization`, `class_weight`	Less transparent; can overfit rare vendors

G. Hyperparameter tuning

- **Split.** Train: CVEs published 2021-01-01 to 2023-12-31. Test: 2024-01-01 to 2025-06-30, untouched until the final evaluation.
- **CV.** Sort training rows by `published`, then `TimeSeriesSplit(n_splits=4)` so every validation fold is later than its training data.
- **Scoring.** `average_precision` (PR-AUC) as the refit metric; also record `roc_auc`.
- **Search.** `RandomizedSearchCV(n_iter=30)` for the boosted model and forest; `GridSearchCV` for Logistic Regression (`C` ∈ {0.01, 0.1, 1, 10}, `penalty` ∈ {l1, l2}, `solver='liblinear'`) and Naive Bayes (`alpha` ∈ {0.01, 0.1, 0.5, 1}).
- **Distributions.** Gradient boosting: `learning_rate` loguniform(0.01, 0.3), `max_leaf_nodes` 15–63, `l2_regularization` loguniform(1e-3, 10). Forest: `n_estimators` 200–600, `max_depth` {None, 10, 20}, `min_samples_leaf` {1, 5, 20}.
- **No contamination.** The TF-IDF vectoriser, imputers and category groupers are steps of the pipeline being searched. Negatives may be down-sampled for speed only inside training folds, never in the test set.

H. Model evaluation

- **PR-AUC** — the main metric, because positives are rare and ROC-AUC looks flattering under heavy imbalance.
- **Recall@top-k% and precision@top-k%** (k = 1%, 5%) — matches an analyst's daily capacity.
- **ROC-AUC** — secondary, for comparison with published work.
- **Brier score and a calibration curve** — needed if the app shows probabilities; use `CalibratedClassifierCV` if poorly calibrated.
- **Baseline comparison** against the Dummy model and the CVSS rule on the same test period.
- **Error analysis.** List the highest-scored negatives and lowest-scored positives; group misses by vendor and CWE.
- **Interpretability.** Logistic coefficients; `permutation_importance` on the test set for tree models.

- **Limitations and label honesty.** KEV includes only vulnerabilities with reliable exploitation evidence and clear remediation guidance relevant to U.S. federal systems, so label 0 means "not confirmed by CISA", not "never exploited". This is a positive-unlabelled problem in practice.
- **Fairness.** No people are involved; the analogous concern is coverage bias toward vendors CISA monitors closely.
- **Drift.** Report test performance by half-year; vendor mix and NVD analysis coverage change over time.

I. Application development (Streamlit)

- **Layout.** Sidebar: model version, training period, test PR-AUC. Main tabs: *Score one CVE*, *Score a batch*, *Model card*.
- **Inputs.** CVE ID (the app fetches the record live from NVD, stripping all `cisa*` fields), or manual entry: description, CWE dropdown, CVSS vector string, vendor. Batch mode: CSV upload with the same columns.
- **Validation.** Regex for `CVE-\d{4}-\d{4,}`; CVSS vector parsed with a strict pattern; clear errors for unknown vendors (treated as "other").
- **Output.** Risk score, its percentile against the test period, a High / Watch / Routine band using thresholds chosen on validation data, and the top contributing features.
- **Visuals.** Score distribution with this CVE marked; batch results as a sortable table and download.
- **Oversight.** A fixed banner: research prototype; the analyst confirms with vendor advisories and threat intelligence before acting.
- **Pipeline loading.** `joblib.load("models/kev_pipeline.joblib")` once with `@st.cache_resource`; the app passes a raw-column DataFrame to `pipeline.predict_proba`, never re-implementing preprocessing.

J. Deliverables

Common list in the *Grading* section, plus: `src/fetch_nvd.py` with resumable paging and logs, `src/build_labels.py`, the vendor-mapping CSV, and a leaky-versus-clean ablation table.

K. Risks and enhancements

- **Failure modes.** Leaving `cisaExploitAdd` in the data (near-perfect but meaningless scores); random splitting; reporting accuracy; NVD rate-limit bans.
- **Disclosures.** Snapshot bias of NVD records; KEV's U.S.-federal scope; missing CVSS for recent CVEs.
- **Deployment blockers.** Needs daily retraining, live threat intelligence, and monitoring; industry already uses FIRST's EPSS for this task.
- **Enhancements.** Compare against EPSS scores as an external benchmark; rebuild publication-day features with NVD's CVE change-history API **[verify]**; add exploit-code availability from public exploit databases for a time-stamped feature.
- **Employer appeal.** Real security workflow, imbalanced learning, text features and leakage control — directly relevant to security-analytics roles.

Viva questions

1. Why can't you simply randomly split CVEs into train and test? What would go wrong?

2. Name two NVD fields you removed and explain how each would leak the label.
3. Your model's PR-AUC is 0.20 while ROC-AUC is 0.95. Explain how both can be true and which one you would report to a CISO.
4. How did you compute `vendor_prior_kev`, and how did you prove it uses no future information?
5. A CVE scored 0.01 was later exploited. Walk through your error analysis and say what feature might have caught it.

Project 2 — Insurance: Flood-claim severity triage at first notice of loss

Teams predict how much the U.S. National Flood Insurance Program will pay on a residential building claim using only what the insurer knows when the claim is first reported, so large losses reach senior adjusters and initial reserves are set sensibly.

A. Project definition

Item	Definition
Title	First-Notice-of-Loss Severity Prediction for NFIP Residential Flood Claims
Business problem	After a hurricane, thousands of claims arrive in days. Insurers must decide which claims need experienced adjusters and how much to reserve before anyone has inspected the house.
Users and stakeholders	Claims managers, reserving actuaries, adjusting firms, reinsurance teams, FEMA's flood program
Why it matters	Mis-routing large losses delays payouts to flooded households; under-reserving distorts the insurer's financial position.
Prediction objective	Estimate the building payment for a claim at first notice of loss (FNOL)
Target variable	`log1p(amountPaidOnBuildingClaim)`, restricted to closed claims with a positive building payment; a secondary binary target `large_loss` = payment ≥ 50% of `totalBuildingInsuranceCoverage`
Inputs at prediction time	Policy and property attributes on file (coverage, flood zone, construction date, floors, elevation status, occupancy, deductible), loss date, location, cause of damage, named flood event
Expected business outcome	A severity score and range per claim; a ranked queue where the top decile captures most large losses

Item	Definition
ML problem type	Regression on a skewed target, plus an optional classification view
Difficulty	Intermediate
Six-week feasibility	Very feasible: one public dataset, no key, clear target; event-grouped validation adds depth

B. Data acquisition

Dataset	Owner	Access	API key	Size (verified)
NFIP Redacted Claims – v3	FEMA (OpenFEMA)	API endpoint `https://www.fema.gov/api/open/v3/NfipClaims`; full-file downloads on the dataset page	No	FEMA states more than 2.7 million claims transactions, updated about monthly
CPI-U index (optional join)	U.S. Bureau of Labor Statistics	Download the annual all-items CPI-U series	Not for small pulls **[verify]**	1 row per year

Important change in 2026. FEMA deprecated the older `FimaNfipClaims` v2 dataset: its data are frozen at 1 June 2026 and FEMA says it will be removed by 15 October 2026. Students must use v3 (`NfipClaims`, without the `FIMA` prefix) and check the v3 data dictionary, because field names below were confirmed from a v2 record **[verify each against v3]**.

Steps.

1. Open the v3 dataset page in a browser (it blocks some scripted fetches) and download the data dictionary.
2. Pull with the API using OpenFEMA query parameters: `$filter` (e.g. `state in ('TX','LA','FL') and yearOfLoss ge 2010`), `$select` for the columns you need, and `$top` / `$skip` paging. Check the per-call record limit in the OpenFEMA API documentation **[verify]**. Alternatively download the full file and filter locally with pandas or DuckDB.
3. Confirmed v2 fields include `dateOfLoss`, `yearOfLoss`, `state`, `countyCode`, `censusTract`, `reportedZipCode`, rounded `latitude` / `longitude`, `ratedFloodZone`, `occupancyType`, `originalConstructionDate`, `originalNBDate`, `numberOfFloorsInTheInsuredBuilding`, `elevatedBuildingIndicator`, `postFIRMConstructionIndicator`, `basementEnclosureCrawlspaceType`, `primaryResidenceIndicator`, `totalBuildingInsuranceCoverage`, `buildingDeductibleCode`, `causeOfDamage`, `floodEvent`, `waterDepth`, `floodWaterDuration`, `buildingDamageAmount`, `amountPaidOnBuildingClaim`, `asOfDate`, `id`.
4. **Authenticity.** Use only `fema.gov`; record `asOfDate`. Cite: "This product uses the FEMA OpenFEMA API, but is not endorsed by FEMA," and follow the OpenFEMA terms and conditions.

5. **Scope.** Residential single-family occupancy (check v3 occupancy codes), `yearOfLoss` 2010–2024, coastal and Gulf states. Use the API's count option to check row counts before downloading.
6. **Join (optional).** Merge annual CPI-U on `yearOfLoss` to convert payments and coverage to constant dollars.
7. **Storage.** `data/raw/nfip_claims_v3_<asOfDate>.parquet` untouched; filters and joins in `src/build_dataset.py`; provenance in `DATA_SOURCES.md`.

Privacy. FEMA redacts policyholder identity; location is generalised (coordinates rounded to one decimal place). Students must not attempt to re-identify properties.

C. Cleaning and data preparation

1. **Missing values.** Elevation fields (`baseFloodElevation`, `lowestFloorElevation`, `elevationDifference`) are mostly null for older or non-certified buildings; keep a missing flag instead of imputing a height.
2. **Encoded missing.** Watch for placeholder codes and nulls in coded fields (e.g. `basementEnclosureCrawlspaceType`, `crsClassificationCode`); confirm each code against the v3 dictionary and map unknowns to `"unknown"`.
3. **Imputation.** Median for numeric, most-frequent or constant for categorical, all inside the pipeline.
4. **Duplicates.** One row per `id`. Check for repeated claim records with identical key fields and payments.
5. **Types and categories.** Parse dates; `ratedFloodZone` has many variants (`A07`, `AE`, `VE`, `X`): build a `zone_group` (A-type, V-type coastal, X/B/C low-risk, other).
6. **Outliers.** Payments cannot exceed coverage; flag rows where they do. Inspect $0 and very small payments separately; zero-payment claims are excluded from the regression target and analysed in EDA.
7. **Encoding.** One-hot for zone group, occupancy, deductible code, cause; target-free frequency grouping for county.
8. **Scaling.** For Ridge and SVR only.
9. **Leakage.** `buildingDamageAmount`, `netBuildingPaymentAmount`, `buildingPropertyValue`, `buildingReplacementCost`, `nonPaymentReasonBuilding`, and every contents or ICC payment field are produced during or after adjustment: drop them. `waterDepth` and `floodWaterDuration` are measured at inspection, so they belong only in a second "post-inspection" model, reported separately.

Filtering rules and the time split run before splitting; imputers, encoders, scalers and any county-level statistics are fitted inside the pipeline.

D. Exploratory data analysis

Question	Chart	Use for feature engineering
How skewed are payments?	Histograms of raw and `log1p` payment	Justifies the log target

Question	Chart	Use for feature engineering
Do V-zone and elevated buildings differ in severity?	Box plots of log payment by zone group and elevation flag	Supports zone grouping and interactions
Does building age matter?	Scatter or binned means of log payment vs age at loss	Motivates `building_age`
How much do events differ?	Bar chart of median payment by top-15 `floodEvent`	Motivates event-grouped CV
Is coverage a ceiling?	Scatter of payment vs coverage with y = x line	Motivates `payment_ratio` view and capping
How do payments change over time?	Line of median real (CPI-adjusted) payment by year	Inflation adjustment and drift check

E. Feature engineering

Feature	How to calculate	Why it may help	At inference?	Leakage risk
`log_coverage`	log1p(`totalBuildingInsuranceCoverage`), CPI-adjusted	Coverage bounds the payment and proxies building value	Yes (policy)	Low
`building_age`	`yearOfLoss` − year(`originalConstructionDate`)	Older construction often floods worse	Yes	Low
`policy_tenure_years`	(`dateOfLoss` − `originalNBDate`) / 365	Long-held policies are typically in long-known flood areas	Yes	Low
`zone_group` × `elevatedBuildingIndicator`	Combined category	Elevation matters most in high-risk zones	Yes	Low
`hurricane_season`	Month of `dateOfLoss` in Jun–Nov	Surge vs rain flooding differ in severity	Yes	Low
`county_hist_median`	Median log payment in that county from earlier years only	Captures local flood exposure	Yes	High if computed on all years; build it inside a custom transformer per fold

Feature	How to calculate	Why it may help	At inference?	Leakage risk
`cause_group`	Grouped `causeOfDamage` codes	Storm surge vs river vs rain	Usually reported at FNOL **[verify]**	Low

F. Model development

Algorithm	Why appropriate	Scaling	Key hyperparameters	Strengths / limitations
Baseline: median log payment per `zone_group` (`DummyRegressor` per group)	Mirrors a simple actuarial table	No	—	Floor to beat
Ridge regression with `TransformedTargetRegressor`	Interpretable linear benchmark	Yes	`alpha`	Misses interactions
Random Forest Regressor	Captures zone × elevation × age interactions	No	`n_estimators`, `max_depth`, `min_samples_leaf`, `max_features`	Memory-heavy on large data; subsample for tuning
`HistGradientBoostingRegressor`	Fast on hundreds of thousands of rows; native missing handling; quantile loss gives ranges	No	`learning_rate`, `max_leaf_nodes`, `min_samples_leaf`, `l2_regularization`	Needs careful early stopping
KNN Regressor (optional)	Similar-claims reasoning adjusters find intuitive	Yes	`n_neighbors`, `weights`	Slow and weak in high dimensions; use on a sample

G. Hyperparameter tuning

- **Split.** Train and validate on `yearOfLoss` 2010–2020; test on 2021–2024 (new events the model has never seen).
- **CV.** `GroupKFold(n_splits=5)` grouped by `floodEvent` (fill missing events with a year-month-state key), so one hurricane never sits in both training and validation folds.
- **Scoring.** `neg_mean_absolute_error` on the log target for refit; also record R^2.

- **Search.** `RandomizedSearchCV(n_iter=25)` for boosting (`learning_rate` loguniform(0.02, 0.3), `max_leaf_nodes` 15–127, `min_samples_leaf` 20–200, `l2_regularization` loguniform(1e-3, 10)); forest on a 200k-row sample (`n_estimators` 200–500, `max_depth` {None, 15, 25}, `min_samples_leaf` {5, 20, 50}); `GridSearchCV` for Ridge `alpha` ∈ {0.1, 1, 10, 100}.
- **No contamination.** CPI adjustment uses an external fixed table; every learned statistic sits inside the pipeline; the 2021–2024 test set is opened once.

H. Model evaluation

- **MAE in dollars** after back-transforming — what a claims manager understands.
- **Median absolute percentage error** — robust to the long tail.
- **R^2 and RMSE on the log scale** — model-comparison metrics.
- **Large-loss capture** — share of `large_loss` claims found in the top 10% of predicted severity.
- **Prediction-interval coverage** — share of test claims inside the 10th–90th percentile range from quantile models.
- **Baselines, error analysis and interpretability** — compare with the zone-median table; break errors down by event, state and zone; permutation importance and partial dependence for age, coverage and zone.
- **Fairness.** Census tract and county correlate with household income and demographics. Check whether under-prediction (which delays senior-adjuster attention) concentrates in lower-income tracts by joining tract-level income from the Census Bureau **[verify]**.
- **Drift.** Risk Rating 2.0 changed NFIP rating from 2021, and occupancy codes changed with it; report performance before and after.

I. Application development (Streamlit)

- **Layout.** Left: claim input form. Right: prediction card, range bar, explanation. Second tab: batch CSV scoring for an event.
- **Inputs.** State, county, flood zone, building coverage, construction year, original policy date, floors, elevated (yes/no), basement type, primary residence, deductible code, cause of damage, date of loss.
- **Validation.** Coverage between 0 and the program maximum; construction year ≤ loss year; required fields enforced; unknown counties mapped to "other".
- **Output.** Point estimate, 10th–90th percentile range, `large_loss` probability and a routing suggestion (senior / standard adjuster).
- **Explanation.** Top drivers for this claim from permutation importance; a chart comparing the estimate with typical claims in the same zone.
- **Oversight.** Banner: the estimate supports triage only; payment is determined by the adjuster's inspection and policy terms.
- **Pipeline loading.** One `joblib` file each for point and quantile pipelines, loaded with `@st.cache_resource`.

J. Deliverables

Common list in the *Grading* section, plus the zone-group mapping table, the event-grouped CV report, and a separate results table for the post-inspection model.

K. Risks and enhancements

- **Failure modes.** Using damage or net-payment fields; random splits that put the same hurricane in train and test; ignoring inflation.
- **Disclosures.** Redacted and generalised locations; NFIP-only (no private flood insurers); payments reflect policy limits, not true damage.
- **Deployment blockers.** Real FNOL systems hold richer data; models need recalibration after each major event.
- **Enhancements.** Join NOAA storm-surge or rainfall data per event; two-part model (probability of payment × severity); add NFIP policy data.
- **Employer appeal.** Insurers and insurtechs in India and abroad run exactly this kind of claims-triage modelling.

Viva questions

1. Why is `buildingDamageAmount` excluded, and in which model, if any, is `waterDepth` allowed?
2. Why did you group CV folds by `floodEvent`? What score would random K-fold give, and why is it misleading?
3. You trained on a log target. How did you convert predictions back to dollars, and what bias does that introduce?
4. How does your quantile model produce a range, and how did you check that the range is honest?
5. Show a claim your model badly under-predicted and explain what information it was missing at FNOL.

Project 3 — Fintech / Banking: Mortgage denial-risk screening with a fair-lending audit

Teams predict whether a home-purchase mortgage application will be denied using the information a lender has at application time, then audit whether the model's errors fall unequally on protected groups — the two questions every bank model-risk team must answer.

A. Project definition

Item	Definition
Title	Mortgage Application Denial-Risk Model and Fair-Lending Audit using HMDA Data
Business problem	Lenders want to spot applications likely to be denied early, so loan officers can request missing documents or suggest alternatives. Regulators and compliance teams want to know whether decision patterns treat groups differently.
Users and stakeholders	Loan officers, credit-risk and model-risk teams, compliance officers, regulators, consumer advocates
Why it matters	Mortgage access drives household wealth; the U.S. Equal Credit Opportunity Act and Fair Housing Act

Item	Definition
	prohibit discrimination, so any model must be explainable and audited.
Prediction objective	For an application that reaches a credit decision, predict the probability it is denied
Target variable	`denied` = 1 if `action_taken` = 3; 0 if `action_taken` ∈ {1 (originated), 2 (approved, not accepted)}
Inputs at prediction time	Loan amount, income, debt-to-income band, combined loan-to-value, property value, loan type, term, lien, preapproval, application channel, AUS used, lender, county and tract income level
Excluded from features, used for audit	`derived_race`, `derived_ethnicity`, `derived_sex`, `applicant_age`, `tract_minority_population_percent`
Expected business outcome	A risk score for early follow-up, and an audit report quantifying group disparities in predictions and errors
ML problem type	Binary classification with moderate imbalance, plus fairness analysis
Difficulty	Intermediate to advanced (messy coded fields, ethics)
Six-week feasibility	Feasible with one state and two years of data

B. Data acquisition

Dataset	Owner	Access	API key	Notes (verified)
HMDA Data Browser and Data Browser API	FFIEC, operated by the CFPB	CSV endpoint `https://ffiec.cfpb.gov/v2/data-browser-api/view/csv?states=..&years=..&actions_taken=..`	No	Loan-level public LAR with derived fields; available from 2018 onward in this schema
Public LAR field definitions and schema	FFIEC / CFPB	Web documentation	—	Field names use underscores, e.g. `denial_reason_1`, `aus_1`

Steps.

1. Pick one state with a manageable volume and two consecutive recent years (e.g. year *Y*−1 for training and *Y* for testing). Confirm the latest published year on the Data Browser **[verify]**.

2. Download with the documented CSV endpoint, filtering `actions_taken=1,2,3`. Further filters such as loan purpose may be available as query parameters **[verify in API docs]**; otherwise filter in pandas.
3. Apply scope rules in code: `loan_purpose` = 1 (home purchase), `lien_status` = 1, `occupancy_type` = 1, `business_or_commercial_purpose` = 2, `derived_dwelling_category` = single-family site-built, `reverse_mortgage` and `open-end_line_of_credit` not applicable.
4. Read the field definitions page with the dataset open, and write a data dictionary that records every coded value you keep.
5. **Authenticity.** Download only from `ffiec.cfpb.gov`. Note the dataset type (snapshot or dynamic) and freeze date shown in the Data Browser FAQ.
6. **Join (optional).** None required; tract-level fields are already appended by FFIEC.
7. **Storage.** `data/raw/hmda_<state>_<year>.csv` untouched; `DATA_SOURCES.md` with URL, query string, download date and dataset version.

Privacy and licence. HMDA public data are modified before release to protect applicants (for example, loan amounts are reported as rounded values) and contain no names or addresses. Treat them as public U.S. government data; never attempt to link records to individuals.

C. Cleaning and data preparation

1. **Missing values.** Compute the share of `NA` and `Exempt` per column; drop columns that are mostly exempt for your state's lenders.
2. **Encoded missing.** `NA`, `Exempt`, and codes `1111` (exempt), `8888` (not applicable) and `9999` (no co-applicant / not provided) are text or numeric placeholders, never real values. Convert them to `NaN` or to an explicit category before any numeric conversion.
3. **Imputation.** Median for `income`, `property_value`, `combined_loan_to_value_ratio`; constant `"missing"` for banded `debt_to_income_ratio`; add missing indicators.
4. **Duplicates.** Public LAR rows have no ID; check exact duplicates across all columns and report them rather than silently dropping.
5. **Types and categories.** `debt_to_income_ratio` mixes bands (`"20%-<30%"`, `">60%"`) with single values (`"36"` … `"49"`): map to an ordered numeric midpoint plus an ordinal band. `income` is in thousands of dollars.
6. **Outliers.** Negative or zero income, CLTV above 200, and extremely large loan amounts: cap at 1st/99th percentiles fitted on training data, and document each rule.
7. **Encoding.** One-hot for loan type, AUS, channel; top-20 lenders by `lei` plus `"other"`.
8. **Scaling.** For Logistic Regression and SVM.
9. **Leakage.** `denial_reason_1–_4` exist only for denials; `interest_rate`, `rate_spread`, `total_loan_costs`, `origination_charges`, `discount_points`, `lender_credits`, `hoepa_status` and `purchaser_type` exist only for originated loans. Even their missingness reveals the label, so drop them before the split.

D. Exploratory data analysis

Question	Chart	Use for feature engineering
How does denial rate change with	Bar chart of denial rate	Ordinal DTI encoding; threshold

Question	Chart	Use for feature engineering
debt-to-income?	by DTI band	features at 43% and 50%
Is CLTV non-linear in its effect?	Binned denial rate vs CLTV	Bins or tree models
Do loan types differ (conventional, FHA, VA, USDA)?	Grouped bars of denial rate	One-hot plus DTI interaction
How much do lenders differ?	Denial rate for top-20 lenders	Lender grouping; warns of lender-mix drift
How do denial reasons distribute?	Bar chart of `denial_reason_1` among denials	Explains what the model must learn (EDA only, never a feature)
What are raw denial rates by race, ethnicity, sex and age group?	Grouped bars with counts	Baseline disparities for the audit

E. Feature engineering

Feature	How to calculate	Why it may help	At inference?	Leakage risk
`dti_mid` and `dti_over_43`	Midpoint of DTI band; flag ≥ 43%	DTI is the leading denial reason in many markets	Yes	Low
`loan_to_income`	`loan_amount` / (`income` × 1000)	Affordability beyond reported DTI	Yes	Low
`cltv_band`	Bins: ≤80, 80–90, 90–95, 95–100, >100	Collateral risk is stepwise	Yes	Low
`has_co_applicant`	From co-applicant fields = no co-applicant code	Second income lowers risk	Yes	Low; check it does not proxy sex
`aus_used`	`aus_1` category (DU, LP, other, none)	Underwriting route affects outcomes	Yes	Low
`tract_income_ratio`	`tract_to_msa_income_percentage`	Neighbourhood income context	Yes	Proxy risk: correlates with race; test with and without

Feature	How to calculate	Why it may help	At inference?	Leakage risk
`lender_group`	Top-N `lei`	Lenders differ in credit box	Yes	Low

F. Model development

Algorithm	Why appropriate	Scaling	Key hyperparameters	Strengths / limitations
Baselines: `DummyClassifier`; rule "deny if DTI > 50% or CLTV > 100"	Shows value beyond simple credit-box rules	No	—	Required reference
Logistic Regression	Industry standard for credit; coefficients support reason codes	Yes	`C`, `penalty`, `class_weight`	Linear unless interactions are added
Decision Tree (depth ≤ 5)	Human-readable rules for audit and explanation	No	`max_depth`, `min_samples_leaf`, `ccp_alpha`	Unstable; lower accuracy
Random Forest	Robust non-linear benchmark	No	`n_estimators`, `max_depth`, `min_samples_leaf`, `max_features`	Less transparent
`HistGradientBoostingClassifier`	Usually strongest on tabular data with missing values	No	`learning_rate`, `max_leaf_nodes`, `l2_regularization`	Needs extra explanation work

G. Hyperparameter tuning

- **Split.** Train and validate on year *Y*−1; test on year *Y*, so the test reflects next year's applications.
- **CV.** `StratifiedKFold(n_splits=5, shuffle=True, random_state=42)` within year *Y*−1.
- **Scoring.** `roc_auc` for refit; also record `average_precision` and `neg_brier_score`.
- **Search.** `GridSearchCV` for Logistic Regression (`C` ∈ {0.01, 0.1, 1, 10}) and Decision Tree (`max_depth` ∈ {3, 4, 5}, `min_samples_leaf` ∈ {50, 200, 500}); `RandomizedSearchCV(n_iter=30)` for forest and boosting with the same style of distributions as Project 1.
- **No contamination.** All mapping, capping, imputation and encoding live in the pipeline; caps are learned from training folds only; year *Y* is opened once.

H. Model evaluation

- **ROC-AUC** — overall ranking ability.
- **PR-AUC** — denials are the minority class.
- **Precision and recall at the chosen threshold** — threshold set on validation data for a follow-up budget (e.g. top 15% of applications).
- **Brier score and calibration plot** — scores are presented as probabilities.
- **Baseline comparison** — against Dummy and the DTI/CLTV rule.
- **Error analysis** — false negatives by loan type and lender; false positives by DTI band.
- **Interpretability** — logistic coefficients and permutation importance; per-application reason codes from the top contributors.
- **Fairness audit (mandatory).** For each group of `derived_race`, `derived_ethnicity`, `derived_sex` and age band, report predicted denial rate, true-positive rate and false-positive rate, and their differences or ratios from the reference group (`fairlearn.metrics.MetricFrame` is allowed). Repeat with `tract_income_ratio` removed to test proxy effects. State clearly that the model reproduces historical decisions, so disparities may reflect past bias as well as unobserved credit factors (credit scores are not in public HMDA).
- **Drift.** Compare feature distributions between years with the population stability index.

I. Application development (Flask or Streamlit)

- **Layout.** Page 1 *Application check*; page 2 *Fairness dashboard*; page 3 *Model card*.
- **Inputs.** Loan amount, income, DTI band, property value, loan type, term, co-applicant (yes/no), AUS, county. No race, ethnicity, sex or age inputs.
- **Validation.** Positive amounts; CLTV computed and checked ≤ 200; DTI chosen from the allowed bands.
- **Output.** Risk band (low / medium / high), calibrated probability, top three contributing factors in plain language (e.g. "DTI above 50%").
- **Fairness dashboard.** Static charts from the test-set audit with group sizes.
- **Oversight.** Banner: not a credit decision system; any adverse action requires human underwriting and legally compliant notices.
- **Pipeline loading.** `joblib.load` of the full pipeline at app start; inputs assembled into a one-row DataFrame with raw column names.

J. Deliverables

Common list in the *Grading* section, plus `fairness_audit.ipynb`, the coded-value mapping table, and a one-page memo addressed to a compliance officer.

K. Risks and enhancements

- **Failure modes.** Keeping `interest_rate` or denial reasons; treating `9999` as a number; using race or age as features.
- **Disclosures.** No credit scores or full credit files; derived race and sex fields are CFPB groupings; rounded loan amounts.
- **Deployment blockers.** Real underwriting needs bureau data, legal review, model-risk validation and adverse-action reason codes.

- **Enhancements.** Compare two states; post-processing threshold adjustment to reduce error-rate gaps and report its accuracy cost; monotonic constraints in `HistGradientBoostingClassifier` (DTI and CLTV should only increase risk).
- **Employer appeal.** Banks and NBFCs hiring in credit risk and model validation look for exactly this blend of predictive work and fairness reporting.

Viva questions

1. Why did you exclude `action_taken` values 4, 5 and 6 from the training data?
2. Give two fields whose missingness alone would leak the label, and explain why.
3. Race is not a feature in your model. Can the model still produce racial disparities? Show evidence from your audit.
4. How did you convert `debt_to_income_ratio` to a numeric feature, and what information did you lose?
5. Your false-positive rate differs between two groups. What would you recommend to the compliance officer, and what is the trade-off?

Project 4 — Healthcare: Non-laboratory screening for undiagnosed prediabetes and diabetes

Teams build a screening model that uses only questions and simple body measurements to flag adults who should get an HbA1c blood test, trained on one national survey cycle and tested on a later one — a genuine external validation students rarely get to do.

A. Project definition

Item	Definition
Title	Non-Laboratory Risk Screening for Undiagnosed Dysglycemia using CDC NHANES
Business problem	Many people with diabetes or prediabetes do not know it. Community health programmes, pharmacies and workplace camps need a quick, cheap way to decide who should be sent for a blood test.
Users and stakeholders	Community health workers, primary-care clinics, public-health programmes, corporate wellness providers
Why it matters	Undiagnosed diabetes goes untreated and causes complications; prediabetes is the window for prevention.
Prediction objective	For an adult with no diabetes or prediabetes diagnosis, estimate the probability that HbA1c is in the prediabetes-or-diabetes range

Item	Definition
Target variable	`dysglycemia` = 1 if `LBXGH` (glycohemoglobin, %) ≥ 5.7, else 0 (5.7% and 6.5% are the American Diabetes Association cut-offs)
Study population	Age ≥ 20; `DIQ010` ≠ 1 (never told diabetes); `DIQ160` ≠ 1 (never told prediabetes); not pregnant; non-missing `LBXGH`
Inputs at prediction time	Age, sex, BMI, waist circumference, blood pressure readings, told-high-blood-pressure, smoking, physical activity, income-to-poverty ratio, education
Excluded from features, used for audit	Race/ethnicity (`RIDRETH3`)
Expected business outcome	A screening tool that matches or beats the U.S. Preventive Services Task Force rule (adults 35–70 with overweight or obesity) at the same number of people referred
ML problem type	Binary classification; optional three-class extension (normal / prediabetes / diabetes range)
Difficulty	Intermediate (multi-file joins, survey design, coded missing values)
Six-week feasibility	Very feasible: small files, no key, rich documentation

B. Data acquisition

Dataset	Owner	Access	API key	Size (verified)
NHANES 2017–March 2020 pre-pandemic files (prefix `P_`)	CDC National Center for Health Statistics	XPT file per component, linked by `SEQN`	No	`P_GHB`: 9,737 non-missing HbA1c values (ages 12+)
NHANES August 2021–August 2023 files (suffix `_L`)	CDC NCHS	Same	No	`GHB_L`, `DEMO_L`, `DIQ_L`, `BMX_L` confirmed

Files to download for each cycle.

Component	2017–Mar 2020	Aug 2021–Aug 2023	Key variables
Demographics	`P_DEMO`	`DEMO_L`	`SEQN`, `RIDAGEYR`, `RIAGENDR`, `RIDRETH3`, `INDFMPIR`, `DMDEDUC2`, `RIDEXPRG`, weights
Glycohemoglobin	`P_GHB`	`GHB_L`	`LBXGH`

Component	2017–Mar 2020	Aug 2021–Aug 2023	Key variables
Diabetes questionnaire	`P_DIQ`	`DIQ_L`	`DIQ010`, `DIQ160`, `DIQ180`
Body measures	`P_BMX`	`BMX_L`	`BMXBMI`, `BMXWAIST`, `BMXHT`, `BMXWT`
Blood pressure exam	`P_BPXO`	`BPXO_L` **[verify]**	Systolic and diastolic readings 1–3 **[verify names]**
BP questionnaire	`P_BPQ`	`BPQ_L`	`BPQ020` (told high BP); note `BPQ050A` was replaced by `BPQ150` in 2021–2023
Smoking, physical activity	`P_SMQ`, `P_PAQ`	`SMQ_L`, `PAQ_L` **[verify]**	`SMQ020`; activity items per codebook

Steps.

1. Download each `.xpt` file and its codebook page from the CDC data page into `data/raw/<cycle>/`.
2. Read with `pd.read_sas(path, format="xport")`.
3. Read each codebook's "Code or Value" table and note skip patterns; confirm the target age group of every variable.
4. **Join.** Left-join every file to the demographics file on `SEQN` within a cycle; never join across cycles (IDs are cycle-specific). Stack cycles only after harmonising column names, with a `cycle` column.
5. **Authenticity.** Use only `cdc.gov`. Record file names, release dates and any revision notes; the 2021–2023 release notes say some components were released later.
6. **Provenance.** `DATA_SOURCES.md` lists every file, its codebook URL and download date.
7. **Licence and ethics.** NHANES public-use files are de-identified and free; users must follow the NCHS data use restrictions, including never attempting to identify a participant **[verify current wording on the Data User Agreement page]**.

C. Cleaning and data preparation

1. **Missing values.** HbA1c is missing for people who did not give blood; drop them from the modelling set and report how many were dropped by age and sex.
2. **Encoded missing.** Questionnaire codes 7 (refused), 9 (don't know), 77, 99, 777, 999 are not real answers. Convert them to `NaN` before treating any column as numeric. Age is top-coded at 80.
3. **Imputation.** Median for BMI, waist and BP; most-frequent for categorical answers; add indicators; all inside the pipeline.
4. **Duplicates.** `SEQN` must be unique within each file; assert it.
5. **Types and categories.** XPT reads numbers as floats, including category codes: cast coded variables to strings or integers explicitly. Average the available BP readings.
6. **Outliers.** Check BMI and BP against physiological ranges; flag rather than delete values that are extreme but plausible.
7. **Encoding.** One-hot for sex, smoking status, education; ordinal for activity level.

8. **Scaling.** For Logistic Regression, SVM and KNN.
9. **Leakage.** `DIQ010`, `DIQ160` and diabetes medication items define the population and must not be features. Lab values that track glucose (fasting glucose, insulin) are excluded because the tool is meant to work without blood tests. `DIQ180` (blood test in past three years) is a behaviour, allowed but tested by ablation.

Survey design note. NHANES over-samples some groups and provides weights. Fit models unweighted, but report test-set prevalence and key metrics with and without the phlebotomy/MEC weights (`sample_weight` in scikit-learn metrics), and explain the difference.

D. Exploratory data analysis

Question	Chart	Use for feature engineering
How does HbA1c vary with age?	Scatter with LOWESS line, or binned means	Age bins or spline-like transforms
Is waist or BMI more informative?	Overlaid density by class; correlation heatmap	Choice of adiposity features; waist-to-height ratio
How does risk relate to blood pressure?	Binned target rate by systolic BP	Hypertension flag
Does income relate to undiagnosed dysglycemia?	Target rate by `INDFMPIR` band	Socio-economic feature and fairness check
Do the two cycles differ?	Side-by-side distributions of each feature and target rate by cycle	Expected drift in the external test
How do groups differ in prevalence?	Target rate by `RIDRETH3` and sex, with counts	Baseline for the fairness audit

E. Feature engineering

Feature	How to calculate	Why it may help	At inference?	Leakage risk
`waist_to_height`	`BMXWAIST` / `BMXHT`	Central adiposity predicts insulin resistance across body sizes	Yes (tape measure)	Low
`bmi_class`	WHO bands, optionally with Asian-specific cut-offs as a sensitivity analysis	Non-linear risk	Yes	Low
`mean_sbp`, `mean_dbp`	Mean of available readings	Hypertension clusters with dysglycemia	Yes (BP cuff)	Low

Feature	How to calculate	Why it may help	At inference?	Leakage risk
`age_band`	20–34, 35–44, 45–54, 55–64, 65+	Mirrors clinical screening rules	Yes	Low
`hypertension_any`	Told high BP or mean SBP ≥ 130	Combines history and measurement	Yes	Low
`inactive`	Derived from activity items	Lifestyle risk	Yes (self-report)	Low
`uspstf_flag`	Age 35–70 and BMI ≥ 25	Baseline rule and possible feature	Yes	Low

F. Model development

Algorithm	Why appropriate	Scaling	Key hyperparameters	Strengths / limitations
Baselines: `DummyClassifier`; USPSTF rule	The current guideline is the bar to beat	No	—	Required
Logistic Regression	Clinically familiar; can become a points-based score	Yes	`C`, `penalty`, `class_weight`	Assumes additive log-odds
Gaussian Naive Bayes	Fast probabilistic benchmark on continuous measures	Yes (helps)	`var_smoothing`	Independence assumption is violated (BMI and waist) — good discussion point
SVM (RBF) with `probability=True`	Effective on small-to-medium tabular data	Yes	`C`, `gamma`	Slow to calibrate; less interpretable
Random Forest / `HistGradientBoostingClassifier`	Captures age × BMI interactions; handles missing values (HGB)	No	As in earlier projects	Risk of overfitting on a few thousand rows; tune `min_samples_leaf`

G. Hyperparameter tuning

- **Split.** Development set: 2017–March 2020 cycle. External test set: August 2021–August 2023 cycle, opened once.
- **CV.** `StratifiedKFold(n_splits=5, shuffle=True, random_state=42)` on the development set (cross-sectional data, so shuffling is acceptable).

- **Scoring.** `roc_auc` for refit; record `average_precision` and `neg_brier_score`.
- **Search.** `GridSearchCV` for Logistic Regression (`C` ∈ {0.01, 0.1, 1, 10}), Naive Bayes (`var_smoothing` ∈ {1e-9, 1e-7, 1e-5}), SVM (`C` ∈ {0.1, 1, 10}, `gamma` ∈ {"scale", 0.01, 0.1}); `RandomizedSearchCV(n_iter=30)` for forest and boosting with `min_samples_leaf` 10–100.
- **No contamination.** Coded-missing conversion is a fixed rule applied to both cycles; imputers, scalers and encoders are fitted inside folds; the 2021–2023 cycle never informs any choice.

H. Model evaluation

- **Sensitivity and specificity at a chosen threshold** — screening is judged on missed cases and unnecessary tests.
- **Sensitivity at the USPSTF rule's referral rate** — a fair head-to-head with the guideline.
- **PPV and NPV** — what a health worker can tell a person.
- **ROC-AUC and PR-AUC** — threshold-free comparison.
- **Calibration plot and Brier score** — needed if probabilities are shown.
- **External validation** — development CV score versus 2021–2023 test score; explain any drop.
- **Error analysis** — false negatives by age band and BMI class (e.g. lean older adults).
- **Interpretability** — logistic coefficients as odds ratios; permutation importance.
- **Fairness** — sensitivity and false-negative rate by `RIDRETH3`, sex and income band, with group sizes and confidence intervals (bootstrap).
- **Generalisation** — NHANES represents the U.S. population. South Asians develop diabetes at lower BMI, so the model must not be used in India without local validation.

I. Application development (Streamlit)

- **Layout.** A single-page form for a health worker, a result card, and an "About this model" expander.
- **Inputs.** Age, sex, height, weight (BMI computed), waist, two BP readings, told high BP (yes/no), smoker status, physical activity level.
- **Validation.** Age 20–80; height 120–220 cm; weight 30–250 kg; BP 70–250 / 40–150 mmHg; warnings for implausible values.
- **Output.** Risk probability, risk band, and the action text "Recommend HbA1c or fasting glucose test" or "Routine advice".
- **Explanation.** The person's three largest risk contributors, and a chart placing their risk on the test-set distribution.
- **Oversight.** Banner: screening aid only; does not diagnose; trained on U.S. data; refer to a clinician.
- **Privacy.** The app stores nothing; no names collected.
- **Pipeline loading.** One `joblib` pipeline including unit conversion, imputation, scaling and the model.

J. Deliverables

Common list in the *Grading* section, plus the cycle-harmonisation table (variable names across cycles) and the external-validation report.

K. Risks and enhancements

- **Failure modes.** Treating code 9 as a number; joining across cycles on `SEQN`; including people already diagnosed; reporting accuracy alone.
- **Disclosures.** U.S. population only; single HbA1c measurement (no repeat confirmation); survey weights not used in training.
- **Deployment blockers.** Requires clinical validation in the target population and regulatory review as a health tool.
- **Enhancements.** Three-class model; add earlier NHANES cycles; build a points-based score from the logistic model; compare with a published non-laboratory risk score.
- **Employer appeal.** Shows multi-source data engineering, clinical metrics and external validation — valued by health-analytics firms.

Viva questions

1. Why did you remove people with `DIQ160` = 1 from the population instead of using it as a feature?
2. What do codes 7 and 9 mean in NHANES questionnaires, and what happens if you forget to convert them?
3. Your CV ROC-AUC was higher than your 2021–2023 test ROC-AUC. Give two reasons from the data.
4. At the same number of people referred, does your model find more cases than the USPSTF rule? Show the numbers.
5. Your sensitivity is lower for one group. Is the cause sample size, a feature, or prevalence? How did you check?

Project 5 — Agriculture: Pre-harvest county corn-yield forecasting from weather

Teams forecast each county's corn yield on 31 August — before harvest — by joining USDA's official county yields with NASA daily weather, and they test the forecast on later seasons the model has never seen. This is the one project that is true forecasting rather than retrospective prediction.

A. Project definition

Item	Definition
Title	End-of-August County Corn-Yield Forecasting for the U.S. Corn Belt using USDA NASS and NASA POWER
Business problem	Grain traders, ethanol plants, crop insurers and agricultural lenders need yield expectations weeks before harvest numbers exist.
Users and stakeholders	Commodity analysts, grain procurement teams, crop-insurance actuaries, farm lenders, extension officers
Why it matters	Heat and drought in July and August can cut yields sharply (2012 is the classic example); early warning changes buying,

Item	Definition
	hedging and loss reserves.
Prediction objective	On 31 August of year *t*, forecast county corn-grain yield for year *t*
Target variable	`yield_bu_ac` from NASS Quick Stats: `CORN, GRAIN - YIELD, MEASURED IN BU / ACRE`, `agg_level_desc = COUNTY`, `source_desc = SURVEY`
Inputs at prediction time	Daily weather 1 April – 31 August of year *t*; the county's yields from years before *t*; county location
Expected business outcome	County forecasts with lower error than the county's recent trend, especially in abnormal-weather years
ML problem type	Regression on panel data (county × year) with strict time ordering
Difficulty	Advanced (two APIs, weather aggregation, time-aware validation)
Six-week feasibility	Feasible if scoped to 4–6 states and the weather pull is finished in Week 2

B. Data acquisition

Dataset	Owner	Access	API key	Notes (verified)
Quick Stats API	USDA National Agricultural Statistics Service	`https://quickstats.nass.usda.gov/api/api_GET/` with parameters such as `commodity_desc`, `statisticcat_desc`, `agg_level_desc`, `state_alpha`, `year`, `format=JSON`	Yes — free, by registration	50,000-record limit per query; split by state or year
NASA POWER daily point API	NASA Langley Research Center	Daily point endpoint with `community=AG`, variables such as `T2M_MAX`, `T2M_MIN`, `PRECTOTCORR`, `ALLSKY_SFC_SW_DWN`, `RH2M`	No	Daily data from 1981; up to 20 parameters per point request; coarse grid (about half a degree)
Census Gazetteer county file **[verify**	U.S. Census Bureau	Text file with county GEOID and internal-point latitude / longitude	No	Used to place one weather point per county

Dataset	Owner	Access	API key	Notes (verified)
link]				

Coverage warning. NASS announced in April 2024 that it would discontinue county estimates for crops, then reinstated them in March 2025, effective for the 2024 crop season. Check in Week 1 that county corn yields exist for every year you plan to use, especially 2023–2025.

Steps.

1. Register for a NASS API key and store it in an environment variable, never in code or GitHub.
2. Pull county corn-grain yield for Iowa, Illinois, Indiana, Nebraska, Minnesota and Ohio, 2000–latest, one state per call to stay under 50,000 rows. Also pull `AREA HARVESTED` for weighting (not as a feature).
3. Understand the schema: `state_ansi` + `county_ansi` form the 5-digit FIPS code; `Value` is text with commas; `CV (%)` gives sampling variability where present.
4. Download the Gazetteer county file and keep each county's internal point.
5. For each county, request NASA POWER daily data for 1 April–31 August of every year in one date-range call. Sleep between calls and cache each response as JSON; NASA uses -999 for missing.
6. **Join.** NASS yields ↔ Gazetteer on FIPS; weather aggregates ↔ yields on FIPS + year.
7. **Storage and provenance.** Raw JSON per county in `data/raw/power/`; NASS pulls in `data/raw/nass/`; `DATA_SOURCES.md` records parameters, extraction dates and the NASS notice "This product uses the NASS API but is not endorsed or certified by NASS." NASS and NASA data are U.S. government public data with no personal information.

C. Cleaning and data preparation

1. **Missing values.** Counties drop out of NASS in some years because of disclosure rules or low response; do not impute the target — drop those county-years and report how many.
2. **Encoded missing.** NASS uses text codes such as `(D)` (withheld to avoid disclosing individual operations) and `(NA)`; NASS's combined record has `county_ansi` empty and `county_name = OTHER (COMBINED) COUNTIES` — remove it. NASA POWER uses -999.
3. **Imputation.** Weather gaps: linear interpolation within a county's season before aggregation (a fixed rule); feature-level median imputation inside the pipeline.
4. **Duplicates.** One row per FIPS × year; check for both `SURVEY` and `CENSUS` records and keep `SURVEY`.
5. **Types.** Strip commas and convert `Value` to float; pad FIPS to five characters.
6. **Outliers.** Very low yields are often real (drought, derecho, flooding). Investigate with news and keep them; they are the cases the model exists for.
7. **Encoding.** State one-hot; county identity is represented by lagged-yield features, not one-hot (too many levels).
8. **Scaling.** For Ridge, SVR and KNN.
9. **Leakage.** No weather after 31 August; no same-year production, harvested area or state yield; lagged yields use only years before *t*; never shuffle years.

D. Exploratory data analysis

Question	Chart	Use for feature engineering
How strong is the technology trend?	Line chart of state-average yield by year	Trend term and lagged-mean features; detrended target as an option
How large are weather shocks?	Distribution of yield anomaly (yield minus 5-year county mean) by year	Shows the hard years (e.g. 2012) the model must catch
Does July heat hurt yields?	Scatter of anomaly vs July days with Tmax > 30 °C	Extreme-heat features around pollination
Is rainfall effect non-linear?	Binned anomaly vs June–August precipitation	Both drought and excess-water terms; trees
Are nearby counties similar?	Map of anomalies for a drought year	Supports state features and spatial error analysis
Is the weather grid too coarse?	Count of counties sharing an identical POWER cell	Explains limits of county-level weather

E. Feature engineering

Feature	How to calculate	Why it may help	At 31 Aug?	Leakage risk
`gdd_may_aug`	Sum of ((min(Tmax, 30) + max(Tmin, 10)) / 2 − 10) per day, floored at 0	Heat units drive crop development	Yes	Low
`hot_days_jul`	Days in July with Tmax > 30 °C (also > 32 °C variant)	Heat at pollination reduces kernel set	Yes	Low
`prcp_jja`, `prcp_jul`	Total precipitation June–August and July	Water availability during grain fill	Yes	Low
`max_dry_spell_jul_aug`	Longest run of days with precipitation < 1 mm	Captures flash drought better than totals	Yes	Low
`srad_jul_aug`	Mean `ALLSKY_SFC_SW_DWN`	Radiation limits photosynthesis	Yes	Low
`lag_mean_5y`	Mean county yield over years *t*−5 to *t*−1	Encodes county productivity and technology	Yes	High if the window includes year *t*
`year_index`	*t* − 2000	Linear technology	Yes	Low; trees

Feature	How to calculate	Why it may help	At 31 Aug?	Leakage risk
		trend		cannot extrapolate it, so pair with `lag_mean_5y`

F. Model development

Algorithm	Why appropriate	Scaling	Key hyperparameters	Strengths / limitations
Baseline: `lag_mean_5y` (or per-county linear trend)	What an analyst would assume with no weather	No	—	Must be beaten in bad years
Ridge regression	Interpretable weather effects; extrapolates the trend	Yes	`alpha`	Misses thresholds such as heat > 30 °C
KNN regression	"Analogue years" reasoning familiar to agronomists	Yes	`n_neighbors`, `weights`	Sensitive to feature scaling and irrelevant features
Random Forest Regressor	Captures non-linear heat and drought effects	No	`n_estimators`, `max_depth`, `min_samples_leaf`, `max_features`	Cannot extrapolate beyond training yields
`HistGradientBoostingRegressor` (point and quantile loss)	Strong tabular learner; quantile loss gives ranges	No	`learning_rate`, `max_leaf_nodes`, `min_samples_leaf`	Same extrapolation limit; mitigate by predicting the anomaly from `lag_mean_5y`

G. Hyperparameter tuning

- **Split.** Train and validate on 2005–2017 (2000–2004 are consumed by the 5-year lag). Test on 2018–latest available.
- **CV.** Expanding-window splits by year: train ≤ 2013 → validate 2014; ≤ 2014 → 2015; ≤ 2015 → 2016; ≤ 2016 → 2017. Implement as a list of `(train_idx, val_idx)` pairs passed to `cv=`. `TimeSeriesSplit` on rows is wrong here because a year contains many counties.
- **Scoring.** `neg_root_mean_squared_error` for refit; also MAE.
- **Search.** `GridSearchCV` for Ridge (`alpha` ∈ {0.1, 1, 10, 100}) and KNN (`n_neighbors` ∈ {5, 15, 30, 60}, `weights` ∈ {uniform, distance}); `RandomizedSearchCV(n_iter=30)` for forest and boosting.
- **Target option.** Predict `yield – lag_mean_5y` (the anomaly) and add the lag back; compare both formulations.

- **No contamination.** Weather aggregation is a fixed calendar rule; lag features are built from earlier years only; imputers and scalers sit in the pipeline; the test years are opened once.

H. Model evaluation

- **RMSE and MAE in bu/acre** — the units traders and insurers use.
- **Skill score versus baseline** — 1 − RMSE_model / RMSE_baseline; positive means weather added value.
- **Error by year** — a bar chart of RMSE per test year; check whether the model captured years with large anomalies.
- **Harvested-area-weighted state yield error** — aggregate county forecasts and compare with NASS state yields.
- **Interval coverage** — share of test county-years inside the quantile model's 10th–90th range.
- **Error analysis** — map residuals for the worst test year; look for unmodelled events (hail, derecho, flooding).
- **Interpretability** — Ridge coefficients; partial dependence of yield on July hot days and precipitation.
- **Fairness** — not about people; check whether small-production or low-response counties have systematically worse errors.
- **Drift** — climate trends, new hybrids and irrigation changes; report error growth across the test years.

I. Application development (Streamlit)

- **Layout.** Sidebar: state, county, year. Main panel: forecast card, weather summary, county history chart, and a state map of forecasts.
- **Inputs.** Pick a county and season from cached data, or upload a CSV of daily weather (date, Tmax, Tmin, precipitation, radiation) for a "what if" season.
- **Validation.** Date range must cover 1 April–31 August; physical ranges for temperature and rainfall; missing days limited to a stated maximum.
- **Output.** Forecast in bu/acre, 10th–90th percentile range, comparison with the 5-year mean ("8% below normal").
- **Visuals.** Cumulative GDD and rainfall versus the county's normal; a Plotly county choropleth of forecast anomalies.
- **Oversight.** Banner: research forecast, not an official USDA estimate; check NASS Crop Production reports.
- **Pipeline loading.** The saved pipeline takes the engineered weather features; the app reuses the same `src/features.py` function used in training to aggregate daily weather, imported, not copied.

J. Deliverables

Common list in the *Grading* section, plus the cached weather archive (or a script that rebuilds it), the feature-calculation module with unit tests for GDD and dry spells, and the per-year error report.

K. Risks and enhancements

- **Failure modes.** Weather after the forecast date; random K-fold across years; NASA rate limits discovered in Week 5; county gaps from the 2024 reporting pause.
- **Disclosures.** Weather at one point per county; no soil, irrigation or management data; NASS estimates have sampling error.
- **Deployment blockers.** Needs in-season forecasts at several dates, satellite vegetation data, and USDA-grade validation.
- **Enhancements.** Forecasts on 31 July and 31 August to show skill gained; soybeans as a second crop; a parallel Indian version using district-level crop statistics from government open-data portals **[verify availability and quality before use]**.
- **Employer appeal.** Joining two public APIs, domain feature engineering and honest time-based validation are exactly what agritech and commodity-analytics teams test for.

Viva questions

1. Why can't you use `TimeSeriesSplit` directly on the rows of this dataset? Show how your folds are built.
2. Calculate GDD by hand for a day with Tmax 34 °C and Tmin 18 °C using your formula.
3. Why do tree models struggle with the technology trend, and how did your anomaly target fix it?
4. Which test year had the largest error, and what happened in that year?
5. Your forecast is made on 31 August. Name one feature you were tempted to add that would have broken this rule.

Comparative analysis and recommendation

The mortgage-denial project (Project 3) scores highest overall, 46 of 50: it combines easy access, a clear business decision and a fairness audit that employers value. NHANES screening is the safest choice for less experienced teams; the cybersecurity and agriculture projects suit the strongest teams. Scores run 1–5, where 5 is best for a six-week student capstone (for data-engineering complexity, 5 means substantial learning at a manageable effort).

Criterion	P1 Cyber (CVE / KEV)	P2 Insurance (NFIP)	P3 Banking (HMDA)	P4 Health (NHANES)	P5 Agriculture (NASS + POWER)
Business impact	5	4	5	4	4
Dataset credibility	5	5	5	5	5
Dataset accessibility	4	4	5	5	3
Data engineering complexity	4	3	3	4	5
Feature engineering opportunities	5	4	4	3	5
Diversity of applicable algorithms	4	4	5	5	4

Criterion	P1 Cyber (CVE / KEV)	P2 Insurance (NFIP)	P3 Banking (HMDA)	P4 Health (NHANES)	P5 Agriculture (NASS + POWER)
Evaluation complexity (learning value)	5	4	5	5	4
Application demonstration potential	4	4	5	4	5
Portfolio value	5	4	5	4	4
Six-week feasibility	3	5	4	5	3
Total (of 50)	**44**	**41**	**46**	**44**	**42**

Why the scores differ.

- **Credibility** is 5 for all five because every source is a primary government publisher (NIST, CISA, FEMA, FFIEC/CFPB, CDC, USDA, NASA).
- **Accessibility** is lower for P5 (registration key, two APIs, the 2024 county-estimate pause), P1 (API key and paging) and P2 (FEMA's 2026 move to v3 and a browser-only dataset page).
- **Data engineering** is highest for P5 (weather aggregation and three-way join) and P1/P4 (nested JSON; multi-file survey joins). P2 and P3 are single-table datasets with messy codes.
- **Feature engineering** is richest in P1 (text, vendor history) and P5 (agronomic weather indices); P4 has fewer candidate features by design.
- **Algorithms.** P3 and P4 suit every algorithm in the syllabus, including Naive Bayes, SVM and shallow trees; P1's sparse text and P2/P5's size narrow the useful set.
- **Evaluation.** P1 (extreme imbalance, label caveats), P3 (fairness audit) and P4 (external validation, clinical metrics) teach the most about evaluation.
- **Feasibility.** P2 and P4 are lowest-risk; P1 and P5 carry API and labelling risk that must be retired in Weeks 1–2.

Recommendation.

1. **Default choice: Project 3 (HMDA).** Simple download, clear target, all syllabus algorithms apply, and the fairness audit gives students a story interviewers remember.
2. **For first-time or weaker teams: Project 4 (NHANES)**, which is small, well documented and includes a real external test set.
3. **For the strongest teams: Project 1 or Project 5**, with a mentor checkpoint at the end of Week 2 confirming the data pull is complete.
4. **For a cohort with several teams**, offer all five and cap each project at two teams so viva answers cannot be shared.

Six-week implementation plan

Every team follows the same weekly gates; a team that misses the Week 2 gate (complete raw data plus a frozen test set) switches to a reduced scope agreed with the mentor rather than falling further behind.

Week	Focus	Measurable milestone (gate)	Evidence the mentor checks
1	Problem framing and data access	Project charter signed off; API keys working; first raw file saved; every **[verify]** item in the team's project checked	`CHARTER.md` (target, population, prediction time), `DATA_SOURCES.md`, first commit by every member
2	Acquisition, cleaning, split	Full raw data pulled by script; cleaning script reproducible from raw; test set (time- or cycle-based) written to disk and frozen	Re-run of `src/` scripts by the mentor; row counts before and after each filter; leakage register listing every dropped column and why
3	EDA and feature engineering	At least five EDA questions answered on training data; feature table with inference-time and leakage notes	EDA notebook with written conclusions under each chart; feature-engineering notebook; unit tests for two custom features
4	Baselines and model development	Baseline plus at least four models in one `Pipeline` framework, scored with the agreed CV scheme	CV results table (mean ± std); one-page note on why the CV scheme fits the data
5	Tuning, final test, interpretation	Tuned models; test set opened once; error analysis, interpretability and fairness/drift analysis done	Search results CSV; final test metrics vs baseline; error-analysis and model-card drafts
6	Application, documentation, defence	Working Streamlit/Flask app loading the saved pipeline; README, data dictionary, model card complete; presentation and viva	Live demo on a fresh environment from `requirements.txt`; 12–15-minute presentation; individual viva

Weekly ritual. A 30-minute stand-up with the mentor where a different team member each week demonstrates the latest notebook or script live, without notes.

Deliverables, grading and assessing genuine understanding

The grade is 90% team work and 10% individual viva, and each student's team mark can move by up to ±10 points on contribution evidence, so no one passes on code they cannot explain.

Common deliverables (all projects)

#	Deliverable	Minimum content
1	Data acquisition scripts	`src/` scripts that rebuild `data/raw/` from the official sources; logging; no keys in code
2	Processed-data documentation	Filters, row counts per step, leakage register

#	Deliverable	Minimum content
3	EDA notebook	≥ 5 questions, chart plus written finding each
4	Feature engineering notebook	Feature table with formula, rationale, inference-time and leakage notes
5	Model training notebook	Baseline plus ≥ 4 models inside `Pipeline` objects
6	Hyperparameter tuning results	Search spaces, CV scheme, results CSV, chosen parameters
7	Evaluation notebook	Test metrics vs baseline, error analysis, interpretability, fairness / drift
8	Serialised pipeline	Single `joblib` file with preprocessing and model; version recorded
9	Working app	Streamlit or Flask app per the project's section I
10	GitHub README	Problem, data sources, how to run, results, limitations, team roles
11	Data dictionary	Every used column: source field, type, coded values, missing handling
12	Model card	Intended use, out-of-scope use, data, metrics, fairness, limitations, owner
13	`requirements.txt`	Pinned versions; app runs in a fresh virtual environment
14	Final presentation	12–15 minutes plus live demo

Grading rubric (100%)

Component	Weight	Full marks require
Problem framing and data acquisition	10%	Correct target and population; reproducible pull from official sources; provenance recorded
Cleaning and leakage control	15%	Coded-missing handling; leakage register; correct split; all learned steps inside the pipeline
EDA	10%	Questions linked to decisions; findings written, not just plotted
Feature engineering	10%	Justified features with inference-time checks; at least one ablation
Modelling and tuning	15%	Appropriate baseline; ≥ 4 suitable models; CV

Component	Weight	Full marks require
		scheme matches the data structure; sensible search spaces
Evaluation and critical analysis	15%	Business-relevant metrics; baseline comparison; error analysis; fairness or drift; honest limitations
Application	10%	Loads the saved pipeline; validates input; explains output; shows limitations and human oversight
Documentation and code quality	5%	README, data dictionary, model card, readable code, tests for custom functions
Individual viva and contribution	10%	Answers project viva questions alone; commit history and stand-up demos show real contribution

Team components total 90%, but the mentor may scale an individual's team mark by up to ±10 percentage points based on contribution evidence, and an individual who cannot explain their own module in the viva scores zero for the matching rubric line.

How mentors assess genuine understanding (not just AI-generated code)

AI assistants are allowed for coding help, provided every use is declared in an `AI_USE.md` file (tool, purpose, what was changed). Mentors then check understanding directly:

1. **Live change requests.** In a stand-up, ask for a small change on the spot — "add a missing-value indicator for this column", "switch CV to five folds" — and watch the student make it.
2. **Explain-this-cell.** Point at a random cell or function and ask what would break if it were removed.
3. **Data-specific questions.** Ask facts only someone who ran the code knows: how many rows were dropped by the leakage filter, the positive rate in the test set, which coded values appeared.
4. **Hand calculations.** One metric or feature calculated on paper from three rows (e.g. precision from a small confusion matrix, GDD for one day).
5. **Decision log.** A dated `DECISIONS.md` recording why each choice was made; reasons must match the EDA evidence.
6. **Commit history.** Each member commits regularly under their own name; large single commits of finished code near deadlines trigger a follow-up viva.
7. **Viva.** Each student answers at least three of the project's five viva questions individually, with follow-up "why" questions.

Red flags. Library calls nobody can explain; metrics that look too good for the problem (often leakage); identical phrasing between README and AI-style boilerplate; inability to rerun the pipeline from raw data.

Verification log and sources

Core access routes, field names and dataset sizes were confirmed on 3 November 2026; the items marked **[verify]** in each project must be confirmed by students in Week 1 because they could not be checked directly or change often.

Project	Verified	Students must verify
1 Cyber	KEV JSON downloaded from the `cisagov/kev-data` mirror: 12 fields, 1,766 entries, catalog version 2026.10.30. NVD API 2.0 endpoint, 2,000-record page limit, rate limits (5 vs 50 requests per 30 s), `hasKev` parameter, CISA fields present in NVD responses	NVD CVE change-history API for publication-day features
2 Insurance	v3 endpoint `NfipClaims`; more than 2.7 million claims; v2 frozen at 1 June 2026 and scheduled for removal by 15 October 2026; field list from a live v2 record	Every field name in the v3 dictionary; occupancy codes; per-call record limit; BLS CPI access
3 Banking	Data Browser API CSV endpoint and parameters; public LAR field names including `denial_reason_1`–`_4`, `aus_1`–`_5`, derived race, ethnicity and sex	Latest published year; loan-purpose filter parameter; exact privacy modifications
4 Health	`P_GHB` has 9,737 non-missing HbA1c values; `GHB_L`, `DEMO_L`, `DIQ_L` (with `DIQ010`, `DIQ160`, `DIQ180`), `BMX_L` and `BPQ_L` exist; HbA1c instrument change in 2021–2023 needed no adjustment per CDC	`BPXO_L`, `SMQ_L`, `PAQ_L` file and variable names; current data-use wording
5 Agriculture	Quick Stats API needs a free key and limits queries to 50,000 records; NASA POWER needs no key and serves daily AG-community data; NASS county estimates were cancelled in April 2024 and reinstated in March 2025 for the 2024 season	Census Gazetteer file link; county coverage for 2023–2025; NASA POWER rate limits

Sources consulted.

- CISA KEV official mirror: github.com/cisagov/kev-data
- NIST NVD: Getting started, Vulnerabilities API, API workflows
- FEMA: NFIP Redacted Claims – v3, FIMA NFIP Redacted Claims – v2 (deprecated), OpenFEMA data sets
- FFIEC / CFPB: Data Browser API, Public LAR data fields, Public LAR schema, Derived data fields
- CDC NCHS: 2021–2023 release notes, GHB_L, DIQ_L, DEMO_L, BPQ_L, P_GHB, P_DIQ
- USDA NASS: Quick Stats API; reinstatement of county estimates reported by DTN, 19 March 2025
- NASA POWER: power.larc.nasa.gov; parameter usage per the nasapower client documentation